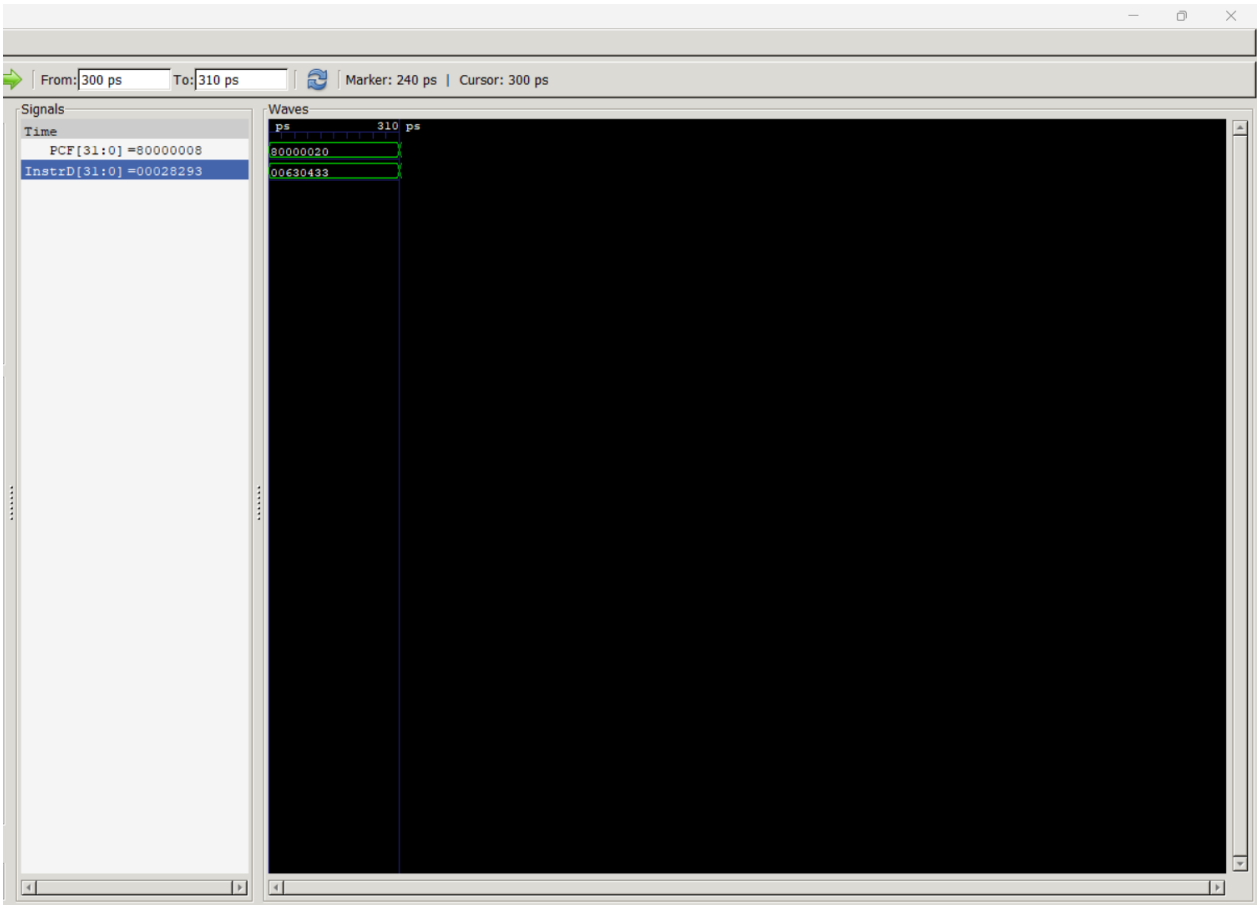
PRELAB 4 REPORT

Ali Ayhan Gunder 2021400219

Rojhat Delibas 2022400057

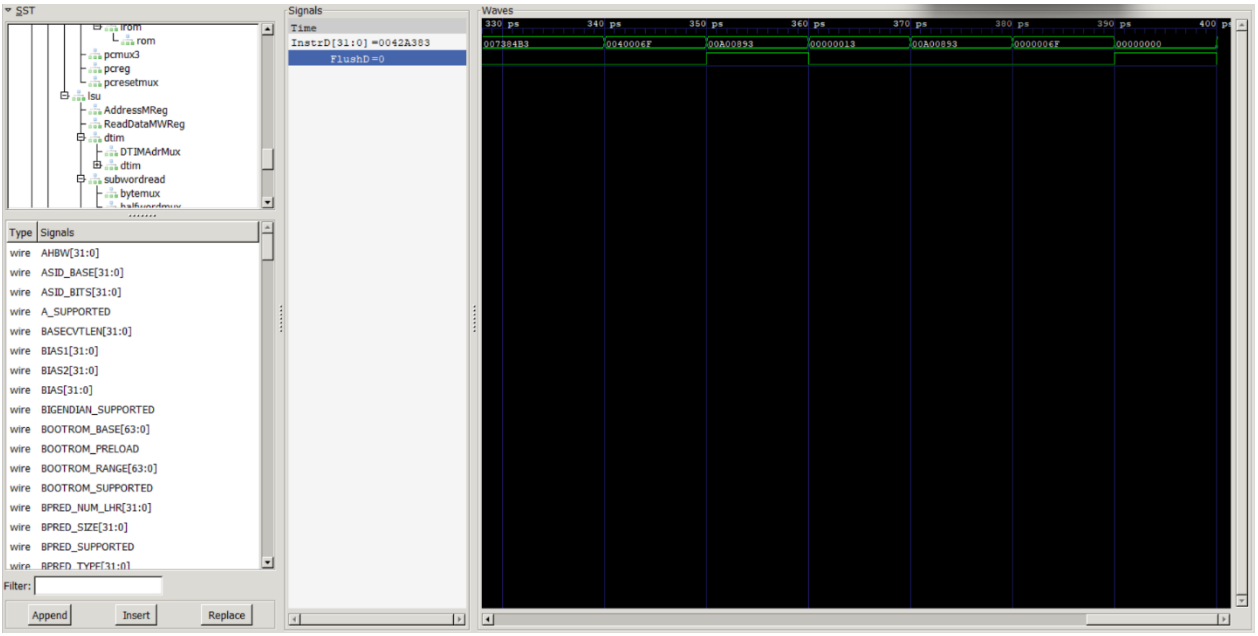
Part I:



Part I Explanation:

When I checked the waveform, I saw that PCF and InstrD were not showing the same instruction at the same moment. For example, PCF was at 0x80000020, but InstrD showed the previous instruction. This is normal in a pipelined processor. Each stage works one cycle later than the one before it. The Fetch stage reads the instruction at the current PC, but the Decode stage is still working on the instruction from the last cycle. Because of this one-cycle delay between Fetch and Decode, InstrD always shows the instruction from one PC earlier. This explains why the values do not match in the same cycle. It also proves that the pipeline is working correctly, since every instruction moves through Fetch → Decode → Execute with a fixed delay.

Part II:



Explanation of the Behavior:

In the waveform, I can see instructions that appear for a short time even though they are not part of the final execution path. This happens when the processor fetches the next instruction before it fully understands that a jump or branch will change the program flow. The Fetch stage always assumes the next instruction is at PC + 4, so it continues reading instructions in a normal sequential order. When the Decode stage later detects that the current instruction is a branch or jump, it realizes that the next instruction already fetched is wrong. Because of this, the processor must remove that incorrect instruction from the pipeline. This behavior is normal in pipelined processors and is used to keep performance high.

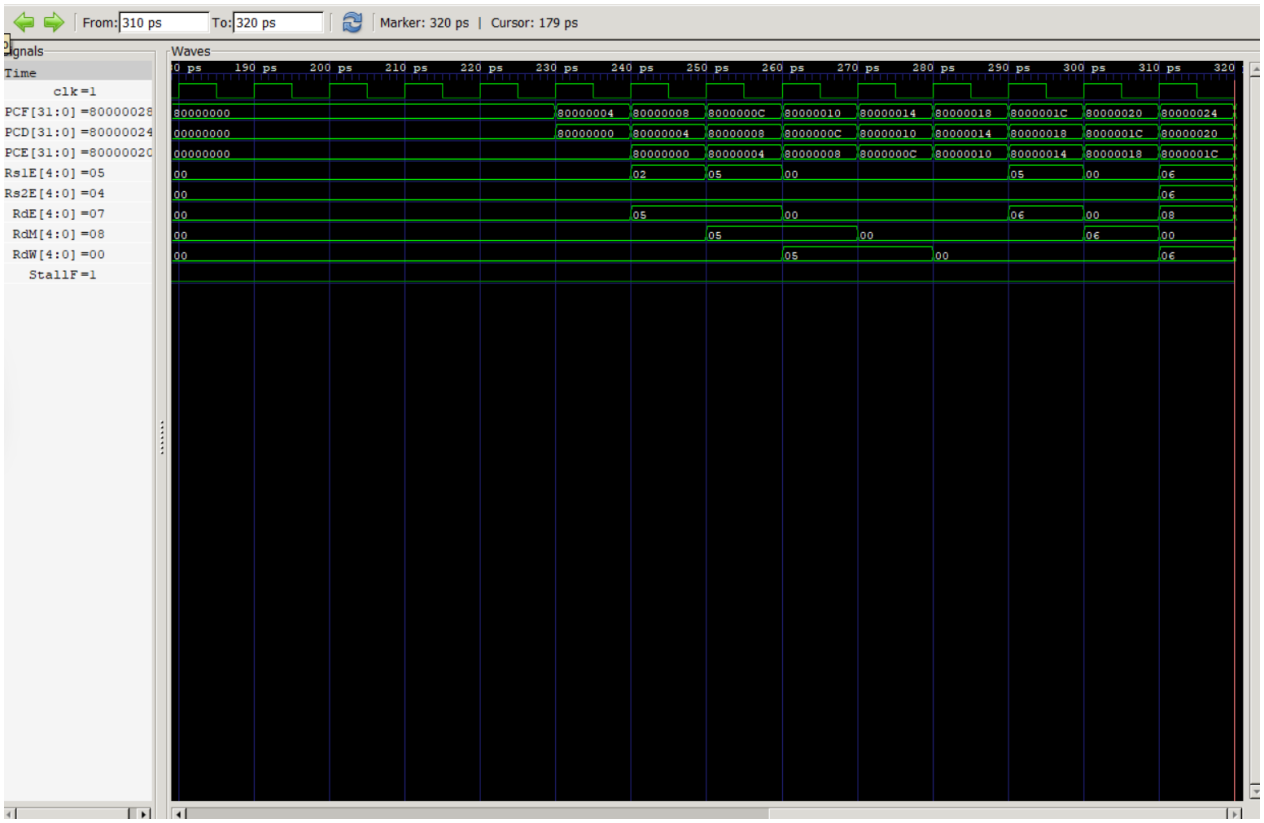
Why This Functionality Is Provided:

If the processor waited for every branch or jump to be fully decoded before fetching the next instruction, execution would slow down a lot. By continuing to fetch instructions early (speculative behavior), the pipeline stays full. Wrong instructions are removed only when needed. This gives better performance because most instructions are sequential, and only a few jumps cause a flush.

What Happens When FlushD Is High:

When the signal **FlushD = 1**, the processor marks the instruction in the Decode stage as invalid. It becomes a **NOP** (no operation) in the next clock cycle. At the same time, the Fetch stage updates the PC to the correct jump target, so from the next cycle it starts fetching the correct instruction. The Execute stage receives a bubble for one cycle, because the wrong instruction is removed. After that, the pipeline continues normally with the correct control-flow path.

Part III:



Part III Answer:

In the waveform, I observe that the Execute stage is using a register value at the same time the Writeback stage is updating that register. Normally this should create a hazard, because the new value is not yet written into the register file. However, pipelined processors solve this problem with a technique called **data forwarding** (also known as *bypassing*).

Data forwarding allows the processor to send the result of an instruction directly from a later pipeline stage back to an earlier one. Instead of waiting until the Writeback stage finishes, the ALU in the Execute stage receives the fresh value through a separate

forwarding path. This makes it possible for an instruction in the Execute stage to use the newest result even if the register file still holds the old value.

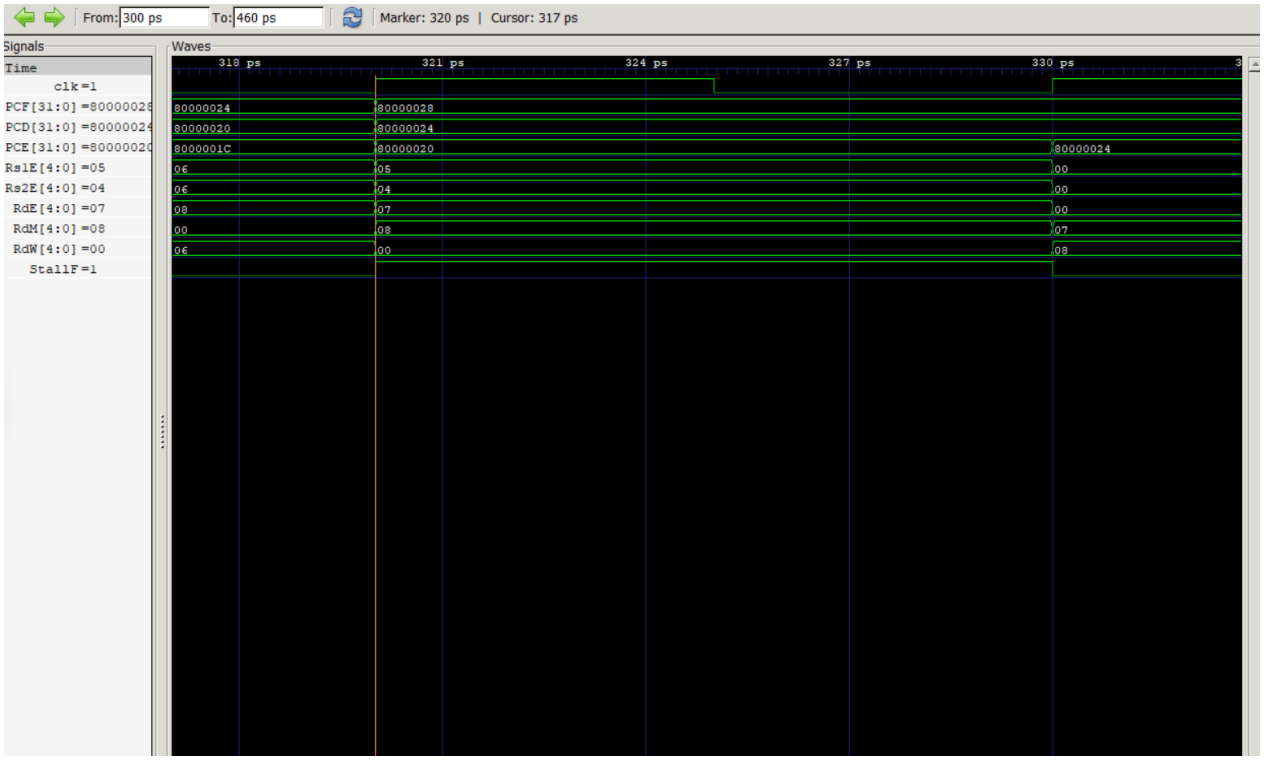
In my waveform, this behavior is visible when Rs1E or Rs2E match the destination register RdW or RdM. The hazard unit detects this match and automatically selects the forwarded value through a multiplexer. Because of that, the ALU gets the correct data without stalling the pipeline. Only load-use hazards still require a one-cycle stall, because the load value becomes available too late to forward immediately.

Data forwarding is important because it improves performance. Without it, many programs would run much slower due to frequent pipeline stalls whenever one instruction depends on another. With forwarding, most dependencies are solved in hardware in the same cycle, keeping the pipeline full and efficient.

References:

Patterson & Hennessy, *Computer Organization and Design RISC-V Edition*, Chapter on Pipelining (Forwarding and Data Hazards).
A reliable online explanation:
https://inst.eecs.berkeley.edu/~cs61c/sp15/lec/09/lec09.pdf (UC Berkeley CS61C Lecture: Pipeline Data Hazards and Forwarding)

Part IV:



Part IV Description:

In this part, I created a simple load-use hazard by placing a load instruction immediately followed by an instruction that uses the loaded register. When the load is in the Execute/Memory stage, the next instruction in Decode needs the same register value. Because the data from memory is not ready yet, the forwarding unit cannot solve this hazard.

The hazard unit detects this condition, and as a result, the signal StallF becomes 1 for one clock cycle. During this cycle, the Fetch and Decode stages stop, and both PCF and PCD keep the same address. A bubble (NOP) is inserted into the Execute stage. In the following cycle, the load completes, the value is available in Writeback, and the dependent instruction can move to Execute using the forwarded result. The stall lasts exactly one cycle, which is the correct behavior for a load-use hazard in the Wally pipeline.